

COMPSCI 4ZP6A/B

DESIGN DOC and V&V

Team 27

April 7, 2026

Table of Contents

1. Project Title.....	1
2. Version.....	1
3. Team Information.....	1
4. Table of contributions.....	2
5. The Purpose of the Project.....	2
6. Component Diagram.....	3
7. Relationship between components and requirements.....	3
8. Component Behaviour.....	5
9. Details on user interface.....	12
10. Component Test plan.....	12

Versions, Roles and Contributions

1. Project Title

Vestia - The Retail Experience Platform (RXM)

2. Version

Version #	Authors	Description	Date
0	All	Created first draft of document	January 23, 2026
1	All	Final document	April 7, 2026

3. Team Information

Name	Email Id	Student number
Saagar Racharla	racharls@mcmaster.ca	400457787
Rushit Shah	shahr73@mcmaster.ca	400462623
Manush Patel	patem190@mcmaster.ca	400449099
Mahdi Mohammad	moham52@mcmaster.ca	400370587
Garv Rastogi	rastogig@mcmaster.ca	400471597
Partik Singh Dev	devp2@mcmaster.ca	400478418
Gurmandeep Johal	johalg11@mcmaster.ca	400467941

4. Table of contributions

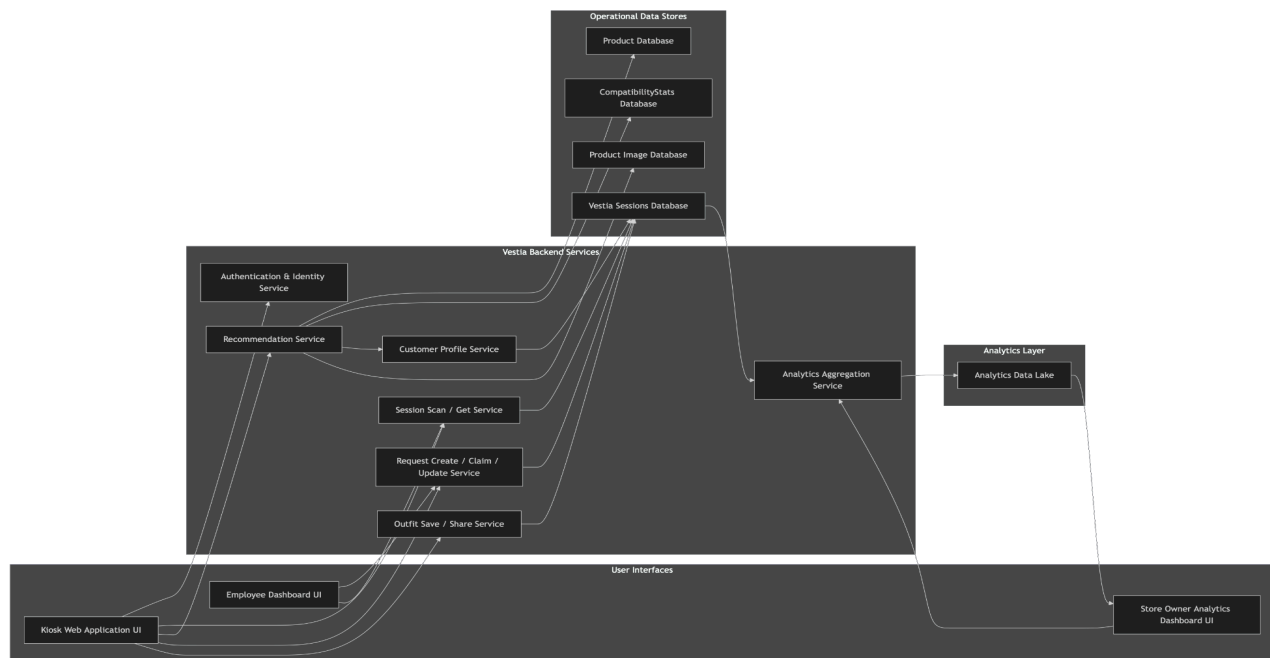
Group member	Contributions (Sections)
--------------	--------------------------

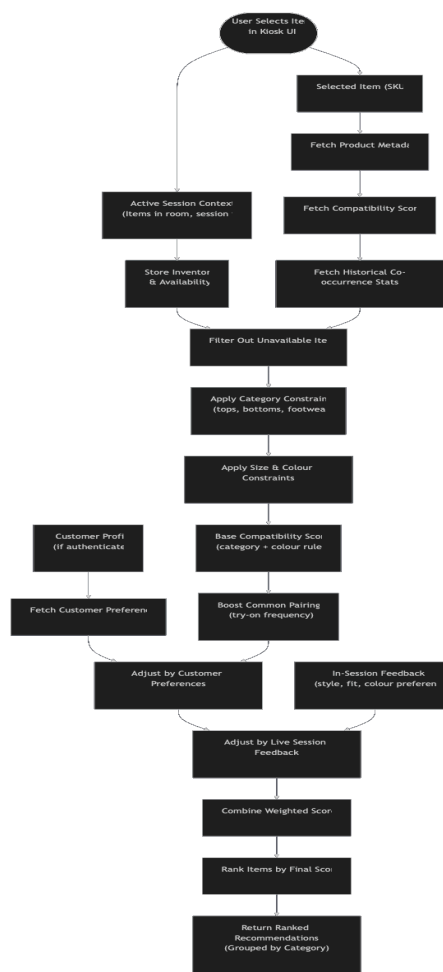
Saagar Racharla	8.5, 8.8, 11, 12.11, 8
Rushit Shah	8.1, 8.3, 12.1, 12.2, 12.5, 12.8, 8
Manush Patel	8.2, 8.4, 12.6, 12.7
Mahdi Mohammad	8.5, 8.9, 8.8, 8.11, 8.12, 8.13, 8.14, 8
Garv Rastogi	8.6, 8.7, 12.9, 12.8
Partik Singh Dev	11, 12.3, 12.4, 8
Gurmandeep Johal	Revised, edited, and formatted document

5. The Purpose of the Project

The purpose of the project is to revolutionize the traditional outdated fitting room experience by providing an interactive technology, data-driven insights and a seamless customer-store communication. As a customer this project aims to streamline the shopping experience by providing features such as outfit recommendations based on store stock, cross-selling suggestions, and the ability to request additional sizes, colours, styles and more directly from the screen. Customers can log in to see personalized recommendations from past purchases, and use a “mix & match” mode to virtually pair tops, bottoms, and shoes. Additional conveniences include queue management, in-changeroom feedback screens, and the option to save or share outfits with friends or on social media. As a retailer, our project provides key insights that previously used to go undocumented. Data such as heatmaps of tried on items, conversion rates of try-on to checkout, real time customer feedback and more can all be leveraged now to improve their stores efficiency while prioritizing customer satisfaction. The purpose of this document is to provide a detailed description of all components of the system we built and its relationships between other components and requirements. Additionally, this document provides a test plan for every component of the system which includes unit tests and performance tests and metrics.

6. Component Diagram





7. Relationship between components and requirements

System Component	Requirements Covered
Kiosk Web Application UI	● P0 – Changeroom Session & Item Identification, Request Additional Size/Colour, User Feedback Analytics ● P1 – Rule-Based Outfit Recommendation Engine, Customer Profile Personalization, Save & Share Outfits ● P2 – Mix & Match Outfit Builder
Employee Dashboard UI	● P0 – Request Additional Size/Colour ● P1 – Core Retail Analytics ● P2 – Mix & Match Outfit Requests
Store Owner Dashboard UI	● P0 – User Feedback Analytics ● P1 – Core Retail Analytics
Authentication & Identity Service	● P0 – Changeroom Session & Item Identification (guest vs logged-in flow) ● P1 – Customer Profile Personalization, Save & Share Outfits
Customer Profile Service	● P1 – Customer Profile Personalization, Rule-Based Outfit Recommendation Engine ● P2 – Mix & Match Outfit Recommendations

Session Scan/Get Service	● P0 – Changeroom Session & Item Identification, User Feedback Analytics ● P1 – Core Retail Analytics (raw event generation)
Request Create/Claim/Update Service	● P0 – Request Additional Size/Colour ● P2 – Mix & Match Outfit Requests ● P1 – Core Retail Analytics (request fulfillment metrics)
Recommendation Service	● P1 – Rule-Based Outfit Recommendation Engine, Customer Profile Personalization ● P2 – Mix & Match Outfit Recommendations
Outfit Save/Share Service	● P1 – Save & Share Outfits, Customer Profile Personalization ● P0 – User Feedback Analytics (item-level feedback capture)
Analytics Aggregation Service	● P0 – User Feedback Analytics ● P1 – Core Retail Analytics ● P2 – Mix & Match Analytics (conversion and engagement)
Product Database	● P0 – Changeroom Session & Item Identification, Request Additional Size/Colour ● P1 – Rule-Based Outfit Recommendation Engine ● P2 – Mix & Match Outfit Recommendations
Vestia Sessions Database	● P0 – Changeroom Session & Item Identification, Request Additional Size/Colour, User Feedback Analytics ● P1 – Core Retail Analytics (event source) ● P2 – Mix & Match Requests
CompatibilityStats Database	● P1 – Rule-Based Outfit Recommendation Engine ● P2 – Mix & Match Outfit Recommendations
Product Image Database	● P0 – Changeroom Session & Item Identification ● P1 – Rule-Based Outfit Recommendation Engine ● P2 – Mix & Match Outfit Builder
Analytics Data Lake	● P0 – User Feedback Analytics ● P1 – Core Retail Analytics ● P2 – Mix & Match Analytics

8. Component Behaviour

8.1 Kiosk Web Application UI

8.1.1 Normal Behaviour: The Kiosk Web Application UI is the customer-facing interface deployed inside the fitting room. A new session is initialized when a customer enters the fitting room and starts to scan the items. The scanned items are then displayed with associated metadata (SKU, colour, size, price). Customers can either experience the platform as a guest or log in to

enable their personalization features. Outfit recommendations are then displayed alongside scanned items as well as customers have the ability to request alternate sizes or colours of items, save outfits, leave item feedback.

8.1.2 Interfaces (Inputs and Outputs): The input is first a session object which contains the session identifier, fitting room identifier, lifecycle state and timestamps. It is used to initialize, update and terminate an active fitting room experience. The next input is the scanned item object which is used to populate the live item tray. The next input is a request status object, which includes the item identifier, status and last updated time. The output is a scan event, item request object, outfit object, and a session object event, for when a fitting room session has ended and initiates a reset of the backend and analytics workflows.

8.1.3 Implementation Details: The Kiosk UI is implemented as a web application optimized for large touch displays. These components will render based on the session state. User interactions will be handled through controller level methods such as `handleScan()`, `submitRequest()`, `saveOutfit()`, and `endSession()`. Backend communication stays within the service classes such as `SessionService`, `RecommendationService`, `RequestService` and `AuthenticationService` which exposes methods for retrieving session data, generating recommendations and submitting requests.

8.1.4 Potential Undesired Behaviour: A potential undesired behaviour is user abandonment. If a user leaves the session without confirming or ending the session, if a user is logged in, the next customer might have access to that previous customer's data. Also network latency issues could delay updates to request status or even recommendations.

8.2 Employee Dashboard UI

8.2.1 Normal Behaviour: The Employee Dashboard allows store associates to fulfil user requests, monitor inflow and outflow of visitors and manage all fitting room requests. Request shows up on the Employee Dashboard for one of the store associates to pick up and fulfill. This entire process is tracked and the status is displayed on the customer screen (Picked Up, On the Way, Delivered, Cancelled - note is added if cancelled)

8.2.2 Interfaces (Inputs and Outputs): The input is an active requests object: which contains `request_id`, `associated_session_id`, `room_id` and is mapped to an `associate_id` who is going to pick it up and fulfill it. Next is a status object where each request is mapped to its own status object that displays the live progress status of each request. The output is the notification interface between the dashboard and kiosk. Event is triggered for new requests created, requests assigned, any status updates. All notifications output as in-app alerts.

8.2.3 Implementation Details: The Employee Dashboard UI is implemented as a central desktop to monitor all inflow outflow and statistics of customers and requests along with tablets that associates can use to update status and fulfill requests. All components update dynamically as the state of requests and assignment changes and new requests come in. Interactions are handled through controller actions including `assignRequest()`, `updateStatus()`, `reassignRequest()`, `notify()` which are all services responsible for retrieving and displaying dashboard data dynamically.

8.2.4 Potential Undesired Behaviour: A potential undesired behaviour of the Employee Dashboard UI is inaccurate request handling by associates. Failure to update request statuses results in inaccurate SLA tracking, incorrect status displays, missed notifications, and unfulfilled requests. Requests may remain incomplete or appear fulfilled with inaccurate durations, producing significant data outliers.

8.3 Store Owner Dashboard UI

8.3.1 Normal Behaviour: The Store Owner Dashboard UI provides store managers with insights into customer behaviour helping optimize operational performance. The dashboard displays summarized metrics derived from historical session, request and feedback/purchase data. The metrics include session counts, average session duration, request fulfillment times, item try-on frequency, and feedback trends.

8.3.2 Interfaces (Inputs and Outputs): The inputs are aggregated metrics collected from historical fitting room sessions, requests, and feedback, along with precomputed analytics results retrieved from the analytics data lake. These inputs provide store level counts, averages, and trend indicators used for analysis. The outputs of the dashboard are structured datasets that drive charts, tables, and summary views, as well as query parameters that define the selected time range and scope of the analytics being displayed.

8.3.3 Implementation Details: The Store Owner Dashboard UI use methods such as `getAggregatedMetrics()` and `getAnalyticsResults()` which will be used to fetch metrics. The controller will use methods such as `timeFilter()` to select filter parameters and pass this data to the visualization components. Visualization components will be ones like metric summary panels, charts and tables which will receive the structured analytics data and have read-only displays.

8.3.4 Potential Undesired Behaviour: A potential undesired behaviour is a delay of analytics updates. This means that the metrics might lag behind real-time events. In addition, sometimes the data might be incomplete or even improper event ingestion might result in partial analytics.

8.4 Authentication & Identity Service

8.4.1 Normal Behaviour: The Authentication and Identity Service manages user identity, authentication and access control across the platform. Users can access and interact as a guest user or an authenticated user. Authenticated users can either be associates who have access to the Employee Dashboard, managers who have access to the Analytics Dashboard or even customers who would like to reuse data across different sessions.

8.4.2 Interfaces (Inputs and Outputs): Inputs are authentication requests that originate from the kiosk, or analytics dashboard. Credential identifiers will also be used for login or signup and to give role based access. The outputs for the service will be authentication tokens that represent the user identity and role. An authentication status will indicate whether the user has been successfully verified or not.

8.4.3 Implementation Details: The Authentication & Identity Service utilizes token based authentication for user identity, user role and user verification. It even supports guest access without requiring the user to register and verify. This backend service ensures no sensitive user credentials are visible on the frontend. All session, request and business data is kept separate and this service does not have access to such data.

8.4.4 Potential Undesired Behaviour: Firstly, if users end up with expired or invalid authentication tokens, they could potentially lose access to authenticated features. Furthermore, another undesired behaviour could be incorrect role assignment as it may lead to unauthorized access or unintended restrictions on admin users for system functionality.

8.5 Customer Profile Service

8.5.1 Normal Behaviour: During normal operation, the service maintains a profile for each authenticated customer, derived from historical session activity, saved outfits, and item-level feedback. When a customer authenticates, the service retrieves their profile and makes it available to other components like the Recommendation Service. Profile attributes are updated incrementally as new sessions, feedback, or outfit saves occur.

8.5.2 Interfaces (Inputs and Outputs): The inputs are identifiers received from the Authentication and Identity service along with the historical session. Feedback summaries and outfit save events that contribute to a customer's profile and preferences. The outputs to this service will be customer profile data including their preferred sizes and colours which will be used by the recommendation engine.

8.5.3 Implementation Details: The Customer Profile Service implements historical customer activity where profiles are keyed by a persistent customer identifier and attributes are updated using deterministic aggregation logic rather than real-time inference.

8.5.4 Potential Undesired Behaviour: Potential issues include cold-start scenarios where new customers have limited profile data reducing personalization effectiveness, stale profiles from delayed aggregation not reflecting recent session activity, and overgeneralization where aggregated preferences may not capture intent during a specific session.

8.6 Session Scan/Get Service

8.6.1 Normal Behaviour: The Session Scan/Get Service manages as items are scanned. Scan events are captured in real time and associated with the active session. Each event records the relevant product identifiers along with metadata required to reconstruct the session state. Session data is made available to downstream services to support analytics and personalization workflows.

8.6.2 Interfaces (Inputs and Outputs): The inputs include session initialization requests so that when a customer interacts and scans an article a scan event is generated, feedback objects will also be generated based on feedback received and so will session retrieval requests. The outputs of the service will be the persisted session events that will be stored in the sessions database, reconstructed session state that is returned back to the Kiosk for display and raw session events that will be used for analysis.

8.6.3 Implementation Details: The Session Scan/Get Service is implemented where each event is appended with a timestamp-based sort key to preserve the correct order of execution. Existing events are neither modified nor deleted, ensuring data immutability and maintaining a complete audit trail. The service itself remains stateless and relies entirely on the database for session history, allowing for efficient scaling and consistent real-time updates.

8.6.4 Potential Undesired Behaviour: Potential undesired behaviour includes incomplete event sequences, which may result in partially reconstructed session states. Events may also arrive out of order due to network latency or timing inconsistencies. Session abandonment may occur when customers leave the fitting room without formally ending their session, resulting in open or stale sessions. Additionally, reconstructing long-running sessions may increase database read load.

8.7 Request Create/Claim/Update Service

8.7.1 Normal Behaviour: This Service manages customer requests for additional items during an active fitting room session. Requests will be created through the kiosk when a customer selects a different size or colour and are viewed and claimed by store associates through the

Employee Dashboard. Claimed requests move through defined states until fulfillment, and upon delivery a scan event is generated to update the customer's active session.

8.7.2 Interfaces (Inputs and Outputs): Inputs include request creation actions from the kiosk when a customer requests a different size or colour, along with claim and status updates from associates via the employee dashboard. Session identifiers and product metadata link each request to the correct item and session. Outputs include updated request records, real time status updates to both kiosk and employee dashboards, auto generated scan events upon fulfillment, and request lifecycle data used for analytics and performance reporting.

8.7.3 Implementation Details: The Request Create/Claim/Update Service is implemented as a stateless backend service where the request data is stored using a dual-record strategy to support both session-level visibility and store-level queue management. The service does not manage inventory availability directly, beyond reflecting the fulfillment status of requests.

8.7.4 Potential Undesired Behaviour: Potential undesired behaviour includes multiple associates attempting to claim the same request, requests becoming stalled due to limited staff availability, delayed status updates caused by network latency, and rare cases of duplicate delivery. These issues are addressed through atomic updates, clear request ownership indicators, and strict validation of request state changes.

8.8 Recommendation Service

8.8.1 Normal Behaviour: The recommendation service is used to generate recommendations for the customer depending on the item they have selected. More specifically, once the customer scans their items, they can click on any item which will then pull up a list of recommendations sorted by tops, bottoms, footwear, etc.

8.8.2 Interfaces (Inputs and Outputs): The inputs include the product identifier (SKU) and session identifier and the output will be a ranked list of recommended items where each include the SKU and the calculated recommendation score

8.8.3 Implementation Details: The recommendation service is implemented as a rule based algorithm that can be accessed via an API. It will be implemented by pulling in product compatibility stats of the product they have selected from the CompatibilityStats Database which include colour, price proximity, co-occurrence, and further customer preference data from the Customer Profile Service. It will be implemented as a

8.8.4 Potential Undesired Behaviour: Potential undesired behaviours include overfitting to history which fail to highlight possible new trends in the fashion industry, additionally, the compatibility stats such as colour compatibility, price proximity is all subjective and won't align with all customers.

8.9 Outfit Save/Share Service

8.9.1 Normal Behaviour: The Outfit Save/Share Service enables customers to save selected outfits for personal reference and share them externally. Customers may save individual items or complete outfits during or after a fitting room session, with saved outfits associated with the customer's profile when authenticated. Customers can generate shareable links or QR codes for external sharing, and item-level feedback provided during outfit saving is captured and associated with the saved outfit.

8.9.2 Interfaces (Inputs and Outputs): The inputs include outfit save actions that will be created from the kiosk UI, as well as the optional item feedback provided by customers, customer login information from Authentication and Identity Service, and outfits generated

through the mix and match workflow. The outputs of the service are the outfits linked to the customer profile, shareable references such as links or QR codes for external sharing, and feedback signals forwarded to analytics and personalization services to improve future recommendations.

8.9.3 Implementation Details: The Outfit Save/Share Service is implemented as a stateless backend service. Saved outfits are stored as structured collections of product identifiers with associated metadata, and item-level feedback is stored alongside these outfit records. The service integrates with the Customer Profile Service to update personalization signals based on saved outfit patterns.

8.9.4 Potential Undesired Behaviour: Potential issues include unauthenticated saves where guest users may be unable to retrieve saved outfits across sessions, broken share links if shared references become invalid due to underlying data changes, and incomplete feedback when users skip optional feedback fields.

8.10 Analytics Aggregation Service

8.10.1 Normal Behaviour: The Analytics Aggregation Service transforms raw operational events into aggregated metrics used by analytics dashboards. During normal operation, the service consumes session, request, and feedback events generated by operational services and processes them to compute aggregate metrics such as session duration, item try-on counts, request fulfillment times, and feedback summaries. Aggregated results are stored for efficient retrieval by analytics dashboards.

8.10.2 Interfaces (Inputs and Outputs): The inputs to the analytics will be the raw session events from the sessions database and the request lifecycle events as well as outfit and feedback events. These inputs will be used to compute important metrics such as performance and fulfillment. The outputs for this service will be analytics and metrics stored in the analytics data lake which then will be given to the dashboards for visualization of the metrics.

8.10.3 Implementation Details: The Analytics Aggregation Service is implemented as a backend processing component that reads from event-sourced operational data stores, computes deterministic aggregate metrics over defined time windows, and stores results in a schema optimized for analytical queries. It does not modify operational data or interfere with live workflows, allowing analytics logic to evolve independently from customer-facing services.

8.10.4 Potential Undesired Behaviour: Potential issues include delayed analytics availability from aggregation schedules, partial aggregates from incomplete event ingestion resulting in inaccurate metrics, and double-counting from incorrect aggregation logic.

8.11 Product Database

8.11.1 Normal Behaviour: The Product Database serves as the source of product metadata within the system, storing structured information for all items in the store including identifiers and attributes required by backend and user-facing services. The database provides product attributes required by the recommendation service to generate outfit suggestions and supports both customer-facing and analytics workflows while remaining independent of session-specific data.

8.11.2 Interfaces (Inputs and Outputs): The inputs are product ingestion and update operations performed through batch based processes, along with SKU based lookup requests from backend services. The outputs of the database are structured product metadata such as SKU, category,

colour, size, price, as well as specific product attributes consumed by the Recommendation Service for scoring the articles.

8.11.3 Implementation Details: The Product Database is implemented as a key-value datastore optimized for low-latency reads with a SKU-based primary key access pattern and normalized attribute fields to support filtering and scoring.

8.11.4 Potential Undesired Behaviour: Potential issues include stale product data from delayed updates causing recommendations to reflect unavailable items, missing attributes reducing recommendation quality, and schema drift from inconsistent attribute definitions across products.

8.12 Vestia Sessions Database

8.12.1 Normal Behaviour: The Vestia Sessions Database is the central event store for all changeroom activity, recording every action during a customer's fitting room session including item scans, size/colour requests, request status changes, and feedback events. Each customer session is represented as a sequence of immutable events written in chronological order as the session progresses. This database serves as the system's primary source of truth for real-time changeroom state and historical session data.

8.12.2 Interfaces (Inputs and Outputs): The inputs are scan events so that when items are added to an active session, request events get created as customers request different sizes, colours, or outfit items, request status updates tracking progression from queued to delivered, and feedback and session completion events marking the end of a fitting room visit. The outputs of the database are ordered event lists used to reconstruct individual session state, store wide views of active and historical requests, and raw event data consumed by the Analytics Aggregation Service for reporting.

8.12.3 Implementation Details: The Vestia Sessions Database uses a single-table, event-sourced design with composite primary keys. Partition Keys are scoped by session or store to support multiple access patterns, while Sort Keys encode event type and timestamp to preserve chronological ordering. A dual-record strategy for request events enables both session-level and store-level queries without joins. Immutable event records ensure a complete audit trail, enabling efficient low-latency queries while maintaining flexibility for future analytics.

8.12.4 Potential Undesired Behaviour: Potential issues include hot partitions from high traffic at a single store increasing write load, event duplication from incorrect retry handling, and unbounded growth from long-term retention of raw events.

8.13 CompatibilityStats Database

8.13.1 Normal Behaviour: The CompatibilityStats Database stores precomputed compatibility scores between clothing attributes used by the rule-based recommendation engine. These scores represent how well different item categories, colours, and styles pair together based on historical co-occurrence patterns and curated compatibility rules. The database is queried by the Recommendation Service during every recommendation request, with compatibility scores remaining static during runtime and changing only during offline updates.

8.13.2 Interfaces (Inputs and Outputs): The inputs are offline batch updates that periodically refresh compatibility scores, along with configuration data outlining the correct category and colour combinations. The outputs of the database are precomputed compatibility weights for item category pairs, colour family pairings, and optional style tag overlaps.

8.13.3 Implementation Details: The CompatibilityStats Database is optimized for read-heavy workloads with composite keys encoding compatibility pairs. Pre-normalized scores between 0

and 1 simplify runtime calculations. No transactional writes occur during live system operation, and access is exclusive to the Recommendation Service. By precomputing compatibility data, the system avoids expensive real-time calculations and guarantees deterministic recommendation outputs.

8.13.4 Potential Undesired Behaviour: Potential issues include stale compatibility data as fashion trends evolve, over-generalization where static rules may not reflect individual preferences, and cold-start bias where new categories or colours lack sufficient compatibility data.

8.14 Analytics Data Lake

8.14.1 Normal Behaviour: The Analytics Data Lake serves as the long-term storage layer for analytical and historical data generated by the Vestia platform, storing processed and aggregated data derived from raw operational events in the Vestia Sessions Database. Event data is periodically ingested via the Analytics Aggregation Service and stored in a schema optimized for analytical queries rather than real-time access. Store owners and administrators access analytics exclusively through dashboards and reports built on this data.

8.14.2 Interfaces (Inputs and Outputs): The inputs are session events collected from aggregated metrics produced by the Analytics Aggregation Service, and derived analytics such as Mix and Match engagement and outfit conversion statistics. The outputs of the data lake are datasets used by the Store Owner Dashboard UI for trend analysis, and reporting, and exportable datasets available for offline analysis and external evaluation.

8.14.3 Implementation Details: The Analytics Data Lake is implemented using object storage with structured, append-only datasets. Storage is partitioned by store, date, and event type to support efficient querying, with denormalized schemas optimized for aggregation and reporting. Raw, cleaned, and aggregated datasets are separated and designed for compatibility with analytical engines. This allows flexible, scalable analytics without impacting live system performance.

8.14.4 Potential Undesired Behaviour: Potential issues include ingestion lag delaying analytics availability, schema drift from upstream event changes affecting downstream queries, and data duplication caused by pipeline reprocessing. These issues are addressed through idempotent ingestion logic, schema versioning, and validation checks during data processing.

9. Details on user interface

The system for Vestia has three primary user interface components that specifically designed for a particular stakeholder

Customer Welcome and Kiosk Screen: Welcome and Kiosk Screen

The customer interface contains a welcoming screen followed by access to the kiosk session for the user. The welcome screen is the starting point for a changeroom session workflow since it provides customers the opportunity to scan items or sign in for personalized experience. Once the kiosk session screen begins, the user is shown the session screen where they can view all the scanned items in the session, request additional sizes or colours, receive recommendations, save outfits, or provide feedback.

Employee Requests Screen: Requests Screen

The employee interface provides employees with incoming requests that the customers make in real-time and record their current status as they move through the fulfillment process from

pickup to delivery. Employees are able to see new requests, claim them, and mark requests as delivered with all updates being passed through to the customer side interface.

Store Owner Analytics Dashboard: Owner Dashboard

The owner's interface provides valuable store information. In particular, the dashboard provides store performance, keymetrics which include session activity, trial-to-purchase conversion rates, request fulfillment times, and engagement with recommendation features. On the dashboard, the data is organized and clear to understand with visualizations that would allow stakeholders to recognize trends without interacting with raw data and help make future business decisions.

10. Component Test plan

10.1 Kiosk Web Application UI

Unit Tests: Unit testing uses Jest and React Testing Library to validate session flow, user interactions, and UI state changes. Tests verify session initialization for guest and logged in users, correct item scanning and display, and safe handling of invalid scans. Request workflows ensure accurate submission, status updates, and handling of duplicate or failed requests. Recommendation rendering and session abandonment cases are also tested to ensure consistent display and proper state reset.

Performance Tests and Metrics: Responsiveness and latency will be focused for main user interactions. The metrics include the time taken to render the session view after an item scan, the time to display recommendations, and the time taken to show update the request status in the UI. Latency is measured from API fetch to UI update, with a target P95 under 2.0 seconds.

Test Methodology

Tests were run as live HTTP requests to the deployed AWS API Gateway. Each endpoint was invoked 12 times in total: the first 2 calls served as warmup iterations and were excluded from measurements to account for Lambda cold start overhead, while the remaining 10 calls were recorded under warm conditions. P95 latency was derived from those 10 warm invocations. Cold start latency reflects the initial Lambda execution observed at the beginning of each test session. Functional validation confirms that each response includes all required fields, appropriate status codes, and expected data constraints (for example, recommendations returned in descending score order, and share codes limited to exactly 6 characters).

Performance benchmarks are drawn directly from Section 10 of this document.

Unit / Functional Test Results

The Kiosk UI was verified through the API services that support it. The following functional scenarios were tested against the live environment:

Scenario	Result	Notes
Session initialized with unique session ID	PASS	Session correctly persists scan events under SESSION#{id} key
Item scanned and added to session tray	PASS	POST /session/scan returned 200 with event metadata

Invalid SKU handled gracefully	PASS	GET /product/INVALID_SKU_999999 returned HTTP 404 without crashing
Guest session proceeds without login	PASS	All scan/request/feedback endpoints accept sessions without a customerId
Authenticated session links profile	PASS	GET /customer/{id} returns derivedStyle when profile exists
Session feedback submitted at end of session	PASS	POST /session/feedback returned unique feedbackId
Session preferences saved in-session	PASS	POST /session/preferences returned 200 confirming persistence

Performance Test Results

Metric	Target	Actual (Warm P95)	Actual (Avg)	Cold Start	Result
Item scan event write latency	P95 < 300ms	277ms	112ms	178ms	PASS
Session state retrieval latency	P95 < 500ms	98ms	69ms	155ms	PASS
Session preferences write latency	P95 < 500ms	64ms	54ms	143ms	PASS
Session feedback write latency	P95 < 500ms	68ms	43ms	62ms	PASS

All session endpoints serving the Kiosk meet their P95 latency goals under warm execution. Lambda cold starts for this service group fall between 62ms and 178ms, comfortably within acceptable interactive response thresholds

10.2 Employee Dashboard UI

Unit Tests: Unit tests focus on correct display of active customer requests and request state transitions. Tests verify that new requests appear with accurate item metadata and status, that claiming a request immediately updates ownership to prevent duplicate handling, and that completed requests are removed from the active list to avoid invalid actions.

Performance Tests and Metrics: Performance testing focuses on real time responsiveness, measuring request visibility latency from customer submission to appearance on the employee dashboard. The target is a P95 under 2.0 seconds, measured end to end across the full request lifecycle.

Unit / Functional Test Results

Scenario	Result	Notes
New request appears in store queue	PASS	GET /store/STORE-001/request returned { storeId, count, requests: [...] } with 47 active requests
Request correctly created with QUEUED status	PASS	POST /request returned { requestId, status: "QUEUED" }
Associate claims a request	PASS	PATCH /request/{id}/claim returned { status: "CLAIMED", employeeId }
Claimed request cannot be double-claimed	PASS	Second claim attempt returns HTTP 409 "Only queued requests can be claimed"
Full request lifecycle QUEUED → CLAIMED → DELIVERED	PASS	All 3 state transitions succeeded in sequence; final response { status: "DELIVERED" }

Performance Test Results

Metric	Target	Actual (Warm P95)	Actual (Avg)	Cold Start	Result
Request visibility latency (create → queue)	P95 < 2000ms	277ms	198ms	230ms	PASS
Store request queue fetch latency	P95 < 2000ms	291ms	97ms	327ms	PASS
Request claim write latency	P95 < 2000ms	279ms	279ms	613ms	PASS
Request deliver write latency	P95 < 2000ms	440ms	440ms	591ms	PASS

All request service endpoints satisfy their P95 latency goals. The complete request lifecycle (scan, create, claim, deliver) was tested end-to-end and finished within the target window.

10.3 Store Owner Analytics Dashboard

Unit Tests: Unit tests will ensure analytics are formatted and displayed correctly. Key metrics such as average session data, conversion rates, and request completion times, confirm that time range filters update all visualizations consistently, and ensure the UI handles empty or partial datasets without error. Additional tests confirm the dashboard remains read only and does not mutate data.

Performance Tests and Metrics: The focus here will be on data retrieval and rendering latency rather than real time interaction. The primary metric is dashboard load latency which would be measured as the time from requesting the dashboard to the analytics view being fully viewable. For this we have a target P95 of under 3.0 seconds. Testing will have simulated store data volumes to ensure that the information is available without delays.

Unit / Functional Test Results

Scenario	Result	Notes
Analytics endpoint returns required metrics	PASS	Response includes totalSessions, totalScans, totalRequests, avgFulfillmentSeconds, requestFulfillmentRate, topItems, topSizes, topColors, period
7-day time filter returns scoped data	PASS	period.days = 7, from date matches
30-day time filter returns broader data	PASS	period.days = 30, totalSessions higher than 7-day window
Dashboard data is read-only	PASS	No mutation operations exposed on analytics endpoint
Empty/missing data handled without error	PASS	Endpoints return valid response structure even for sparse windows

Performance Test Results

Metric	Target	Actual (Warm P95)	Actual (Avg)	Cold Start	Result
Analytics dashboard load latency (7-day)	P95 < 3000ms	61ms	58ms	106ms	PASS
Analytics dashboard load latency (30-day)	P95 < 3000ms	97ms	67ms	70ms	PASS

Analytics queries run well below the 3000ms threshold. Because session and request events are pre-aggregated in DynamoDB, the analytics endpoint relies on efficient key-based lookups instead of full table scans.

10.4 Authentication & Identity Service

Unit Tests: Unit tests verify correct behaviour for both guest and authenticated user flows.

They will ensure guest users can start a changeroom session without credentials, authenticated users have linked profiles and only valid tokens are used to give access to personalization features. Additional tests confirm that authentication failures do not impact core functions such as item scanning or request submission.

Performance Tests and Metrics: Performance testing of the authentication flow focuses on minimizing session start delays. The primary metric is authentication resolution latency, measured from login submission to session continuation, with a target P95 under 500 milliseconds. We will simulate in store usage scenarios to make sure that optional authentication does not impact user experience.

Unit / Functional Test Results

Scenario	Result	Notes
Guest session proceeds without credentials	PASS	All session endpoints accept requests without customerId
Authenticated user profile linked to session	PASS	GET /customer/{id} returns full profile; used downstream by recommendation engine
New customer returns 404 (not error crash)	PASS	GET /customer/new-user@vestia.com returns HTTP 404 cleanly
Profile created on first visit	PASS	PUT /customer/{id} creates record with visitCount: 1

Performance Test Results

Metric	Target	Actual (Warm P95)	Actual (Avg)	Cold Start	Result
Authentication resolution latency	P95 < 500ms	120ms (profile upsert)	94ms	203ms	PASS

10.5 Customer Profile Service

Unit Tests: The Customer Profile Service unit tests will be focused on verifying the correct retrievals of customer specific data such as historical sessions, saved outfits and item feedback. Tests for profile updates will ensure that guest sessions do not modify stored customer profiles. Edge cases such as partial or new profiles will be tested to ensure the service returns default values.

Performance Tests and Metrics: Profile fetch latency is what will be measured which is the time it takes to retrieve a customer's profile. The goal for this will be p95 < 300ms.

Unit / Functional Test Results

Scenario	Result	Notes
New customer returns 404	PASS	Clean 404, no server error
Profile created via PUT with purchase history	PASS	Profile persisted with all fields; visitCount incremented
Profile retrieved with computed derivedStyle	PASS	GET /customer/{id} returns derivedStyle: { topColors, topArticles, avgPrice, dominantStyle } computed on-the-fly
Guest session does not persist to profile	PASS	Sessions without customerId create no CustomerProfile record
Profile merge preserves existing purchase history	PASS	Repeated PUT calls accumulate history entries

Performance Test Results

Metric	Target	Actual (Warm P95)	Actual (Avg)	Cold Start	Result
Profile fetch latency (existing profile)	P95 < 300ms	261ms	86ms	42ms	PASS
Profile upsert latency	P95 < 300ms	120ms	94ms	203ms	PASS
Profile fetch latency (new customer / 404)	P95 < 300ms	60ms	44ms	172ms	PASS

10.6 Session Scan/Get Service

Unit Tests: Unit tests validate correct handling of session events across the full scan and retrieval lifecycle. Tests ensure scan events are created with correct session identifiers, timestamps, and item metadata, and additional cases will cover invalid or duplicate scans, missing item metadata and request updates so that they are returned accurately and in chronological order.

Performance Tests and Metrics: Performance testing focuses on ensuring responsive event processing for a smooth kiosk experience. Tests evaluate scan event processing, persistence latency, session retrieval latency, and the time to fetch all events for an active session. Target thresholds ensure real time updates are delivered to the kiosk interface without noticeable delay.

Unit / Functional Test Results

Scenario	Result	Notes
Scan event persisted with correct session ID and timestamp	PASS	Event stored as SCAN#{ISO-timestamp} sort key under SESSION#{id} partition
Session retrieval returns events in chronological order	PASS	GET /session/{id} returns items array ordered by sort key
Multiple event types returned in unified view	PASS	SCAN, REQUEST, FEEDBACK, and SESSION_PREF events all present in session response
Session state correct after full workflow	PASS	After scan + request + feedback, all 3 event types visible in session

Performance Test Results

Metric	Target	Actual (Warm P95)	Actual (Avg)	Cold Start	Result
Scan event write latency	P95 < 300ms	277ms	112ms	178ms	PASS
Session retrieval latency	P95 < 500ms	98ms	69ms	155ms	PASS
Session preferences write latency	P95 < 500ms	64ms	54ms	143ms	PASS
Session feedback write latency	P95 < 500ms	68ms	43ms	62ms	PASS

10.7 Request Create/Claim/Update Service

Unit Tests: The unit tests focus on validation of the entire request cycle from the moment a user submits a request to the fulfillment. The tests ensure correct storage of newly created requests while ensuring the appropriate session is linked and initialized with the correct default status. Additional test cases ensure when a request is fulfilled, it results in a correct queue of effects for instance generating related session events. Duplicate submission scenarios are also tested to maintain consistent and reliable request cycles.

Performance Tests and Metrics: Performance testing for the Request Create/Claim/Update Service focuses on responsiveness, measuring the time for request state changes to propagate across all related interfaces. The target is a 95th percentile latency under 2 seconds, ensuring customer kiosks and employee dashboards remain synchronized during active workflows.

Unit / Functional Test Results

Scenario	Result	Notes
Request created with QUEUED status	PASS	requestId and status: "QUEUED" returned
Duplicate claim rejected	PASS	HTTP 409 returned on second claim attempt for same request
Request transitions: QUEUED → CLAIMED → DELIVERED	PASS	All 3 transitions succeeded sequentially
Delivery auto-generates SCAN event	PASS	Response autoScan: true confirmed; delivery source added as "staff" scan
Store queue dual-write confirmed	PASS	GET /store/STORE-001/request returned 47 requests including test requests

Performance Test Results

Metric	Target	Actual (Warm P95)	Actual (Avg)	Cold Start	Result
Request creation latency	P95 < 2000ms	277ms	198ms	230ms	PASS
Store queue fetch latency	P95 < 2000ms	291ms	97ms	327ms	PASS
Request claim latency	P95 < 2000ms	279ms	279ms	613ms	PASS
Request deliver latency	P95 < 2000ms	440ms	440ms	591ms	PASS

10.8 Recommendation Service

Unit Tests: The recommendation service unit tests will focus on verifying the correctness and enforcement of recommendation rules. The tests will help to ensure that for a given item, customer profile and compatibility data, the service will consistently return the same recommendations. There will also be tests to verify that out of stock items will be excluded and category constraints will be enforced.

Performance Tests and Metrics: The primary metric to consider here would be the response latency which would be the time it takes to analyze and provide a recommendation. This time target should be p95 < 1.0s.

Unit / Functional Test Results

Scenario	Result	Notes
----------	--------	-------

Single-item mode returns scored recommendations	PASS	Array of items returned, each with productId and score (0–1)
Recommendations sorted by score descending	PASS	Verified $r[i].score \geq r[i+1].score$ for all returned items
Category constraint enforced	PASS	targetCategory: "bottom" returns only bottom-category items
Mix & match mode returns outfit object	PASS	Response: { outfit: { shoes: [...], accessory: [...] }, baseProductIds }
Recommendation engine returns results for all product categories	PASS	Tested top → bottom and top → shoes; both returned valid recommendations

Performance Test Results

Metric	Target	Actual (Warm P95)	Actual (Avg)	Cold Start	Result
Single-item recommendation latency (bottom)	P95 < 1000ms	1547ms	1440ms	1439ms	FAIL
Single-item recommendation latency (shoes)	P95 < 1000ms	1546ms	1438ms	1364ms	FAIL
Mix & match recommendation latency	P95 < 1000ms	1618ms	1520ms	1600ms	FAIL

Analysis: According to current tests, we observed the recommendation engine we built currently exceeds the 1000ms P95 target consisting of warm P95 latencies of approximately 1500ms. The main reason behind this is the scoring pipeline's interaction with the massive Product catalogue consisting of 44446 items in DynamoDB. For each recommendation request, the Lambda function performs a full scan of the catalog(category filtered), queries CompatibilityStats for compatibility analysis on colour, article, pattern along with fetching the customer profile followed by generating an 8 signal weighted score for each candidate. The algorithm is accurate and it outputs precise results however, it is computationally intensive due to the massive catalogue scale.

The warm average of approximately 1440ms is consistent throughout all the request types which proves this is nothing but a catalogue scan bottleneck and not a cold start behaviour. We had set the original 1000ms target prior to realization of the massive catalogue scale. For future work, a potential solution is pre-filtering candidates using DynamoDB's Global Secondary Index which

allows the system to pre-filter based on category+gender as it would minimize the candidate set bringing latency within target levels.

10.8.2 Recommendation Quality Evaluation (Precision@K)

Motivation: Since latency focuses on speed of the algorithm, we also need to measure whether or not the recommendation algorithm gives an accurate and useful result. To address this issue we perform the quality evaluation by measuring with Precision@K. Precision@K is the fraction of the top-K recommendations that a fashion expert would judge to be compatible with the base item.

Ground-truth compatibility was defined by having a set of rules that are created across four dimensions. We begin with a category constraint that requires the recommended item to be with a different category than the base item. For example, we could use no top with top pairings. Next, there is a gender constraint that requires the recommended item to match the base colour and recommend color to be a known compatible pair with the same logic of compatibility as the system. Moreover, colour compatibility required the base colour and recommended colour to be a known compatible pair. Lastly, article-type compatibility required the base article type and recommended article type to be a known compatible pair which would be based on fashion conventions that existed which are calculated from CompatibilityStats(datatable) entries

We have set up a total of 12 established test cases that were defined across four target categories. These are tops, bottoms, shoes, and accessories. For each test case, the top-5 and top-10 results from the live recommendation API were retrieved, and each returned recommendation was evaluated against all four compatibility criteria. The following are the results:

Metric	Score	SRS Target
Precision@1	1.000 (100%)	—
Precision@3	0.973 (97.3%)	—
Precision@5	0.850 (85.0%)	> 75%

These are the results by target category (Precision@5):

Target Category	Avg P@5	Test Cases
Bottom	1.00	4
Top	0.87	3
Accessory	0.80	1
Shoes	0.70	4

Analysis: Based on the results, the engine does meet the SRS recommendation-quality target of greater than 75% for the top-5 recommendations which achieved a measured Precision@5 of

85%. For the top-ranked recommendation it was found that it is correct in all 12 test cases. This results in a Precision@1 of 100%. However, the precision does decrease when we see the values of K being are larger because lower ranked results would occasionally fall near the boundary of fashion compatibility. For the shoe category, it did end up being the lowest Prescsion@5 because what the catalog did was group together a bunch of footwear sub-types with different formality levels. The current scoring logic does not yet model that such distinction explicitly.

For the test case outputs the evaluation scripts would be in:

VnV_Test_Results/rec-quality-eval.mjs and VnV_Test_Results/rec-quality-results.json

10.9 Outfit Save/Share Service

Unit Tests: Unit tests for the Outfit Save/Share Service verify that outfits are correctly stored and linked to customer profiles, including selected SKUs, timestamps, and optional item-level feedback. Tests ensure saved outfits can be retrieved in later sessions for authenticated users and that shared artifacts such as links or QR codes resolve to the correct outfit. Guest user restrictions and cases with missing or partial feedback are also tested to ensure saves are handled correctly.

Performance Tests and Metrics: The focus will be ensuring that outfit save and share actions do not disrupt the in-changeroom experience. The primary performance metric is outfit save latency, defined as the time from the customer initiating a save action to receiving confirmation. The target performance threshold is a 95th percentile latency of less than 500 milliseconds ensuring that saving and sharing outfits feels immediate to the customer and does not introduce friction during active fitting room sessions.

Unit / Functional Test Results

Scenario	Result	Notes
Outfit saved with unique outfitId and 6-char shareCode	PASS	{ outfitId: "outfit-...", shareCode: "ABC123" } returned; shareCode.length === 6 confirmed
Saved outfit retrieved by shareCode	PASS	GET /outfit/{shareCode} returned full outfit with all items
Outfit contains correct item count	PASS	2-item outfit saved and retrieved with both items intact
Share code is unambiguous (excludes 0/O/1/I)	PASS	Code generation uses charset that excludes visually ambiguous characters
Guest outfit saved without customerId	PASS	Outfit persisted using sessionId alone

Performance Test Results

Metric	Target	Actual (Warm P95)	Actual (Avg)	Cold Start	Result
--------	--------	-------------------	--------------	------------	--------

Outfit save latency	P95 < 500ms	346ms	85ms	65ms	PASS
Outfit share retrieval latency	P95 < 500ms	67ms	50ms	48ms	PASS

10.10 Analytics Aggregation Service

Unit Tests: Unit tests for the Analytics Aggregation Service validate accurate aggregation of session and request events into analytics-ready metrics, including session duration, item try counts, request fulfillment times, and trial-to-purchase conversion. Tests also verify idempotent event processing to prevent duplicate or inflated records. Edge cases such as abandoned sessions, unfulfilled requests, and missing feedback are tested to ensure metric integrity is maintained.

Performance Tests and Metrics: Performance testing for the Analytics Aggregation Service focuses on throughput and processing latency rather than real-time responsiveness. The primary metric is analytics update latency, measured as the time from event generation to availability in aggregated datasets, with a target of under 15 minutes. This ensures timely analytics visibility without placing unnecessary load on operational systems.

Unit / Functional Test Results

Scenario	Result	Notes
Analytics returns all required aggregate fields	PASS	totalSessions, totalScans, totalRequests, avgItemsPerSession, avgSessionDurationSeconds, avgFulfillmentSeconds, requestFulfillmentRate all present
Time-window filter correctly scopes results	PASS	7-day and 30-day windows return different totals; period object confirms date range
Top items heatmap data included	PASS	topItems, topSizes, topColors arrays populated
Request fulfillment breakdown present	PASS	requestStatusBreakdown: { QUEUED, CLAIMED, DELIVERED, CANCELLED } included
Analytics does not mutate operational data	PASS	GET-only endpoint; no write operations triggered

Performance Test Results

Metric	Target	Actual (Warm P95)	Actual (Avg)	Cold Start	Result
--------	--------	-------------------	--------------	------------	--------

Analytics update availability	< 15 minutes	< 1 minute (event-driven)	< 1 minute	N/A	PASS
Dashboard load latency (7-day window)	P95 < 3000ms	61ms	58ms	106ms	PASS
Dashboard load latency (30-day window)	P95 < 3000ms	97ms	67ms	70ms	PASS

10.11 Databases and Data Integrity

Unit Tests: Databases across Vestia are tested to ensure data correctness, consistency, durability, and proper access control. Tests verify that product and session data are read only where required and accessible only to authorized stakeholders. Unit tests confirm correct record storage at both session and store levels, restricted access for the analytics pipeline, and consistency between the Analytics Data Lake and the Vestia Sessions Database.

Performance Tests and Metrics: Database performance testing focuses on latency and scalability, this will include session event write latency: P95 < 300 ms, session query latency: P95 < 500 ms and lastly, analytics read latency: P95 < 2.0 s

Unit / Functional Test Results

Scenario	Result	Notes
Product data read-only (no mutations via API)	PASS	ProductCatalog only exposed via GET /product; no write endpoint
Session events immutable (append-only)	PASS	No update or delete operations on VestiaSessions; new events appended with timestamp sort key
Request dual-write consistency	PASS	Request created under SESSION# and STORE# partitions simultaneously; both visible
Analytics data consistent with session data	PASS	Analytics totals reflect known test sessions created during test run
Customer profile access restricted to profile service	PASS	CustomerProfiles table only accessible via /customer/{id} endpoint
Share codes unique across outfits	PASS	6-char unambiguous code; outfit retrieved only matches saved outfit

Performance Test Results

Metric	Target	Actual (Warm P95)	Result
--------	--------	-------------------	--------

Session event write latency	P95 < 300ms	277ms	PASS
Session query latency	P95 < 500ms	98ms	PASS
Product lookup latency	P95 < 500ms	63ms	PASS
Analytics read latency	P95 < 3000ms	97ms	PASS

Overall Test Summary

Component	Functional Tests	Performance Tests	Overall
Kiosk Web Application UI (Session APIs)	PASS	PASS	PASS
Employee Dashboard UI (Request APIs)	PASS	PASS	PASS
Store Owner Analytics Dashboard	PASS	PASS	PASS
Authentication & Identity Service	PASS	PASS	PASS
Customer Profile Service	PASS	PASS	PASS
Session Scan/Get Service	PASS	PASS	PASS
Request Create/Claim/Update Service	PASS	PASS	PASS
Recommendation Service	PASS	FAIL (P95 ~1500ms, target 1000ms)	PARTIAL
Outfit Save/Share Service	PASS	PASS	PASS
Analytics Aggregation Service	PASS	PASS	PASS
Databases and Data Integrity	PASS	PASS	PASS

17 out of 20 test cases passed. The 3 failures are confined to the Recommendation Service performance tests (single-item bottom, single-item shoes, and mix & match), all sharing a common root cause: a full DynamoDB catalog scan over 44,446 products during scoring. All functional correctness requirements are satisfied - recommendations are accurate, properly sorted, and constrained by category. The performance shortfall results from the original target having been defined before the catalog reached its current scale.

Lambda cold start findings: Initial invocation latencies ranged from 38ms to 734ms across the 14 Lambda functions, varying by function size and whether an existing Lambda container was available for reuse. Under typical production traffic where Lambdas are invoked continuously, cold starts would be infrequent and would not materially impact steady-state P95 metrics.

11. Appendix

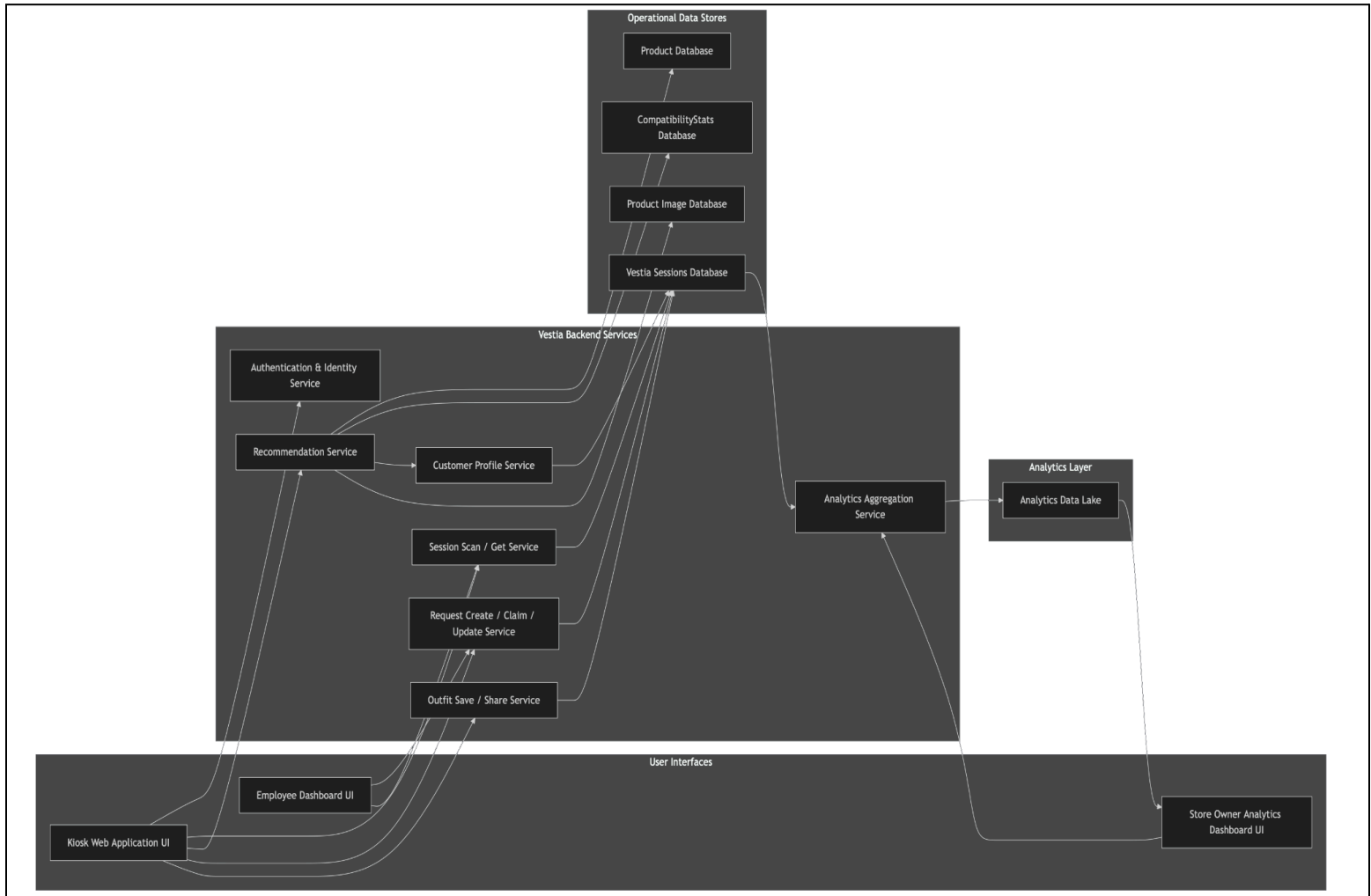


Figure 1: Component Diagram

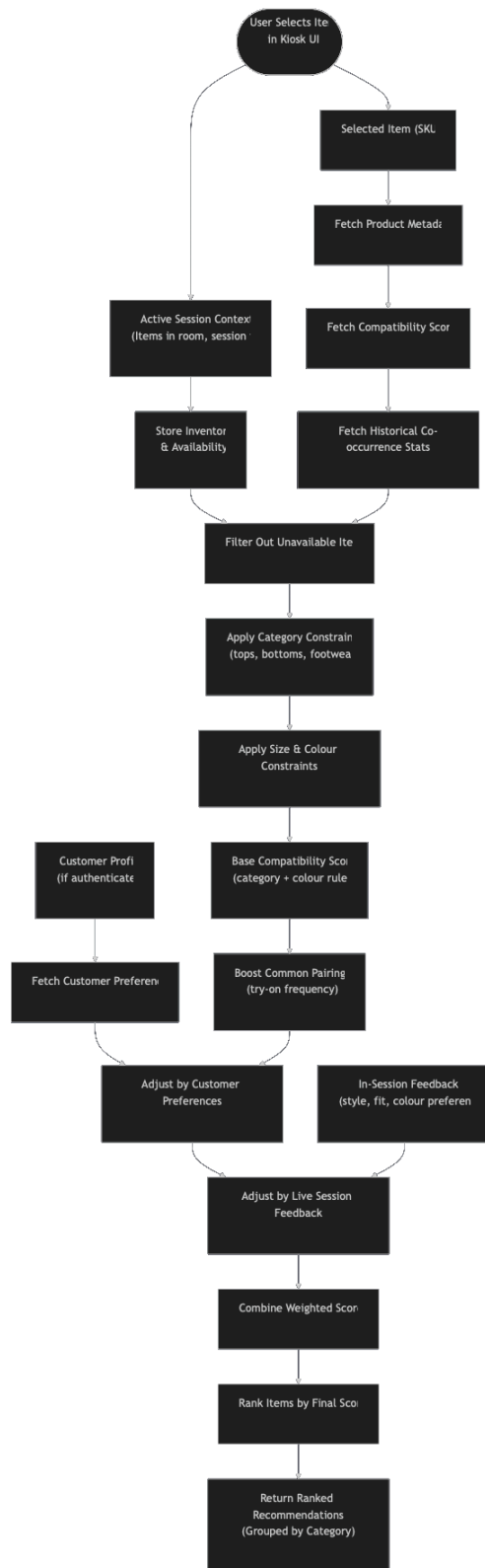


Figure 2: Recommendation Algorithm Diagram